

SARVAJANIK UNIVERSITY

Sarvajani College of Engineering and Technology

Adaptive Traffic Light Controller

Design & Implementation Using Verilog HDL on FPGA

SUBJECT: VLSI Design | CODE: BTEC13404 | SEM: IV | 2025–2026

ET24BTEC003 – Siya Bhalala siyabhalala.ec24@scet.ac.in	ET24BTEC011 – Khushi Desai khushidesai.ec24@scet.ac.in
ET24BTEC019 – Haiya Patel haiyapatel.ec24@scet.ac.in	ET24BTEC034 – Jharna Nakrani jharnanakrani.ec24@scet.ac.in

Branch: Electronics & Communication Engineering

Guide: Prof. (Dr.) Nehal N. Shah | Prof. Chintan H. Panchal | Prof. (Dr.) Ketki C. Pathak

Abstract

This paper presents the design and implementation of an Adaptive Traffic Light Controller for a 4-way road junction using a Moore Finite State Machine (FSM) modeled in Verilog HDL. The controller is simulated and synthesized using Xilinx Vivado and deployed on the Basys 3 FPGA development board (Xilinx Artix-7 XC7A35T). The design implements an 8-state FSM cycling through Green and Yellow phases for each of the four directions — North, East, South, and West — with traffic-density sensor inputs enabling adaptive timing. Each direction is represented by a 3-bit output signal (RED=100, YELLOW=010, GREEN=001). The adaptive extension uses active-LOW density sensors per direction: the controller extends the Green phase if vehicles are detected, and transitions to the next direction otherwise. The design is verified through functional simulation in Vivado, and hardware demonstration is achieved via a custom PCB with LED traffic signals driven by FPGA GPIO pins. The system ensures safe, conflict-free, and efficient traffic flow at a 4-way junction with real-time, fully reprogrammable FPGA-based control.

Keywords — *traffic light controller; Verilog HDL; FPGA; FSM; Moore machine; adaptive control; Basys 3; VLSI design*

I. Introduction

Traffic signal management at road intersections is a fundamental challenge in modern urban infrastructure. Inefficient or manual control systems contribute to congestion, accidents, and wasted resources. Automating traffic light control using digital hardware provides a reliable, scalable, and reconfigurable solution that eliminates human error and responds consistently to real-world conditions.

This project focuses on designing an Adaptive Traffic Light Controller for a 4-way junction using a Finite State Machine (FSM) modeled in Verilog HDL. The controller manages signal sequencing for four directions — North, East, South, and West — cycling through Green and Yellow phases in a fixed round-robin order. An adaptive extension incorporates traffic-density sensor inputs that allow the Green phase to be held longer when vehicles are detected, ensuring more efficient use of road capacity.

The design is synthesized and deployed on the Basys 3 FPGA development board using Xilinx Vivado, and a custom-designed PCB with LED traffic lights demonstrates the real-world hardware prototype.

II. Problem Statement and Motivation

A. Problem Statement

Design an Adaptive Traffic Light Controller for a 4-way junction. Draw a state diagram and state table, write Verilog code to demonstrate results on hardware using FPGA, and make a custom PCB for demonstrating the scenario appropriately.

The controller must:

- Manage signal sequencing at a 4-way road junction safely using a Finite State Machine

- Implement the design in Verilog HDL and verify functionality using simulation waveforms
- Deploy on FPGA and demonstrate with a custom-designed PCB that physically models the junction
- Incorporate traffic-density sensor inputs for adaptive Green phase timing

B. Importance of the Problem

Urban traffic congestion costs billions in lost productivity annually. Manual signal systems are error-prone and inefficient. FSM-based automated controllers ensure fairness and reduce accidents, while FPGA implementation provides real-time, reliable operation. Key motivations include:

- Urban traffic congestion and its economic impact
- Limitations and risks of manual and timer-based signal systems
- Need for adaptive, sensor-driven signal control
- Advantages of FPGA-based reconfigurable hardware implementations

C. Real-World Applications

The design principles demonstrated in this project apply to several real-world scenarios:

- Smart City Traffic Management: Centralized FSM-based control for optimizing signal flow across entire urban networks
- Highway On-Ramp Metering: Automated controllers that regulate vehicle entry onto highways to prevent congestion
- Emergency Vehicle Preemption: Dynamic signal override to clear intersections for ambulances, fire engines, and police
- Pedestrian-Aware Adaptive Control: Sensor-driven signal adjustment that accounts for pedestrian crossing requests and flow

III. System Architecture

The Adaptive Traffic Light Controller is structured around three primary hardware components that work in concert to deliver safe and efficient signal control:

A. State Register

A 3-bit register stores the current FSM state, updated on every positive clock edge. On assertion of the asynchronous reset signal, the register is cleared to State S0 (North Green).

B. Timer / Counter

A 32-bit counter increments every clock cycle. State transitions are triggered when the counter reaches the GREEN_CHECK threshold (200,000,000 cycles at 100 MHz = 2 seconds) or the YELLOW_TIME threshold (200,000,000 cycles = 2 seconds). The counter resets to zero on each state transition.

C. Output Decoder

A combinational always block implements the output logic. By default, all four directions are set to RED. The active direction's signal is then overridden to GREEN or YELLOW based on the current state. This pure

Moore architecture guarantees that outputs depend only on the current state, eliminating glitches from input transitions.

D. Final Working Sequence

The complete operational flow of the system proceeds as follows:

- **FPGA Clock Starts:** 100 MHz system clock begins incrementing the 32-bit counter
- **Counter Generates Timing:** Counter reaches GREEN_CHECK or YELLOW_TIME threshold, triggering a state transition
- **FSM Selects Current State:** The state register is updated to the next state in the $N \rightarrow E \rightarrow S \rightarrow W$ cycle
- **Output Decoder Activates Lights:** Combinational logic maps the current state to the appropriate R/Y/G output signals
- **System Cycles Continuously:** The FSM loops indefinitely, with adaptive hold logic extending Green phases when density sensors are active

PCB Demonstration: A custom PCB was designed to represent the 4-way junction. LEDs on the PCB represent traffic lights controlled by FPGA GPIO outputs, with current-limiting resistors for each signal pin.

IV. FSM Design: State Diagram and State Table

A. FSM Type

The controller uses a Moore FSM in which all outputs depend solely on the current state and are independent of inputs. This guarantees glitch-free, stable signal outputs at all transitions. The FSM has 8 states cycling in a fixed sequence: $NG \rightarrow NY \rightarrow EG \rightarrow EY \rightarrow SG \rightarrow SY \rightarrow WG \rightarrow WY \rightarrow NG$.

B. State Diagram

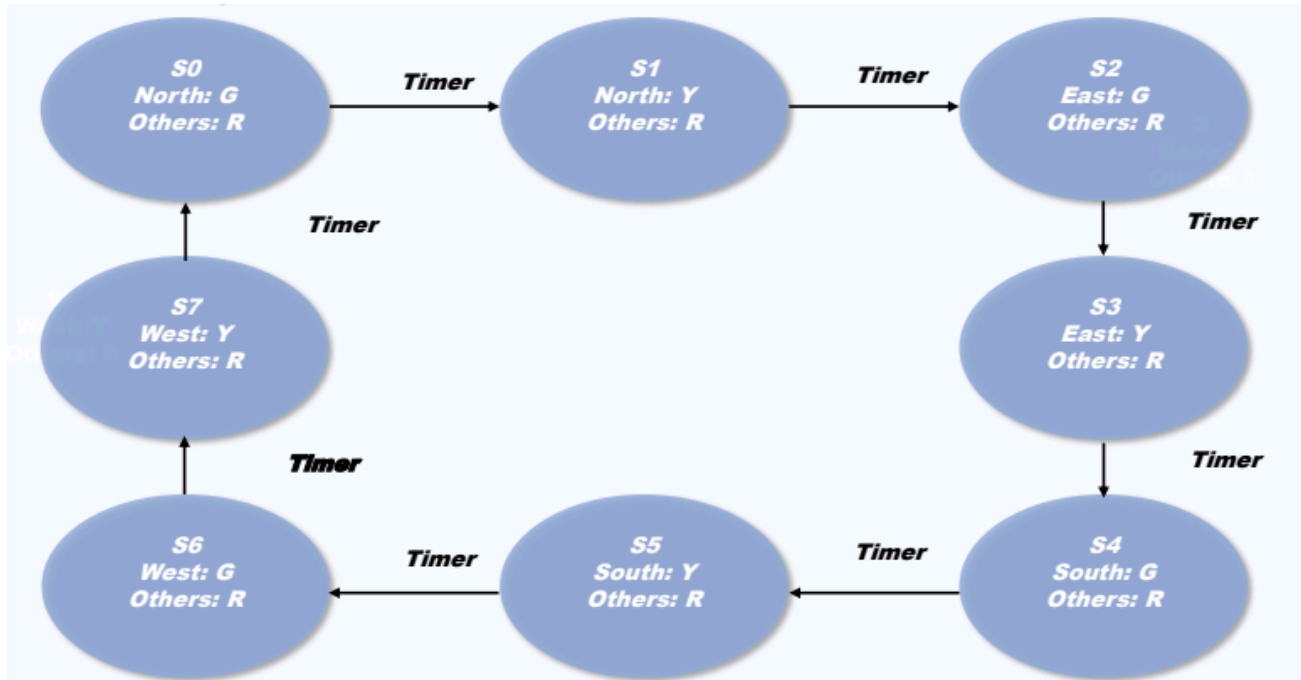


Fig. 2. 8-State Moore FSM State Diagram – Timer-triggered transitions cycling $N \rightarrow E \rightarrow S \rightarrow W$

C. State Table

State	NORTH	EAST	SOUTH	WEST	Next State
S0 (NG)	GREEN	RED	RED	RED	S1 (NY)
S1 (NY)	YELLOW	RED	RED	RED	S2 (EG)
S2 (EG)	RED	GREEN	RED	RED	S3 (EY)
S3 (EY)	RED	YELLOW	RED	RED	S4 (SG)
S4 (SG)	RED	RED	GREEN	RED	S5 (SY)
S5 (SY)	RED	RED	YELLOW	RED	S6 (WG)
S6 (WG)	RED	RED	RED	GREEN	S7 (WY)
S7 (WY)	RED	RED	RED	YELLOW	S0 (NG)

Table I. State Table for 4-Way Traffic Light Controller FSM

V. Verilog HDL Implementation

A. Module Structure

The design is implemented as a single Verilog module Traffic_Light with the following interface:

- Inputs: clk, reset, north_density, east_density, south_density, west_density

- Outputs: north[2:0], east[2:0], south[2:0], west[2:0] — 3-bit one-hot encoded R/Y/G signals
- Parameters: RED=3'b100, YELLOW=3'b010, GREEN=3'b001; state labels NG=0 through WY=7
- Timescale: 1ns/1ps; Clock: 100 MHz (Basys 3 system clock)

B. Sequential Block (State Register and Counter)

The sequential always block, sensitive to posedge clk and posedge reset, handles state transitions and counter management. On reset, the FSM returns to state NG and the counter clears to zero. On each clock edge, the counter increments. When the counter reaches GREEN_CHECK (200,000,000), the controller checks the active density sensor: if no vehicles are detected (active-LOW sensor is HIGH, meaning vN/vE/vS/vW is 0), it advances to the Yellow state; otherwise it resets the counter and remains in Green. Yellow states always transition to the next direction's Green after YELLOW_TIME cycles.

C. Combinational Block (Output Decoder)

A separate combinational always @(*) block implements pure Moore output logic. All four outputs are initialized to RED at the start of every evaluation, and the current state overrides the active direction's output. This clean separation of sequential (timing) and combinational (output) logic is a best-practice HDL design pattern that ensures synthesis produces optimal hardware.

D. Adaptive Density Logic

Traffic density sensors are connected as active-LOW inputs. Internal wires vN, vE, vS, vW are derived by inverting the sensor inputs. During a Green state, if the sensor for the active direction reads HIGH (vehicle present, sensor LOW → inverted to HIGH), the controller resets the counter and remains in Green. If no vehicle is detected, the FSM advances to Yellow. This makes the controller truly adaptive, extending throughput for busier lanes automatically.

VI. XDC Constraints File and Pin Mapping

The design targets the Digilent Basys 3 board (Xilinx Artix-7 XC7A35T). All I/O ports use the LVCMOS33 standard at 3.3V. Configuration properties are set to CFGBVS=VCCO and CONFIG_VOLTAGE=3.3V.

Signal	Package Pin	I/O Standard
clk	W5	LVCMOS33
reset	U18	LVCMOS33
north[0..2]	J1, L2, J2	LVCMOS33
east[0..2]	G2, H1, K2	LVCMOS33
south[0..2]	H2, G3, A14	LVCMOS33
west[0..2]	A16, B15, B16	LVCMOS33
north_density	K3	LVCMOS33
east_density	M3	LVCMOS33
south_density	M1	LVCMOS33

west_density	N1	LVC MOS33
--------------	----	-----------

Table II. XDC Pin Mapping for Basys 3 (Artix-7) Target Board

VII. RTL Schematic

The RTL schematic generated by Xilinx Vivado reveals the synthesized hardware structure of the Traffic Light Controller. The key components visible in the schematic are:

- RTL_ADD: 32-bit adder implementing counter incrementation
- RTL_REG_ASYNC: Asynchronous-reset flip-flop register for the 32-bit counter and 3-bit state
- RTL_GEQ: Greater-than-or-equal comparator for counter threshold comparison
- RTL_MUX: Multiplexers for next-state selection based on density inputs
- RTL_INV: Inverters for active-LOW density sensor inversion (vN, vE, vS, vW)
- RTL_ROM: Read-only memory blocks implementing the combinational output decoder

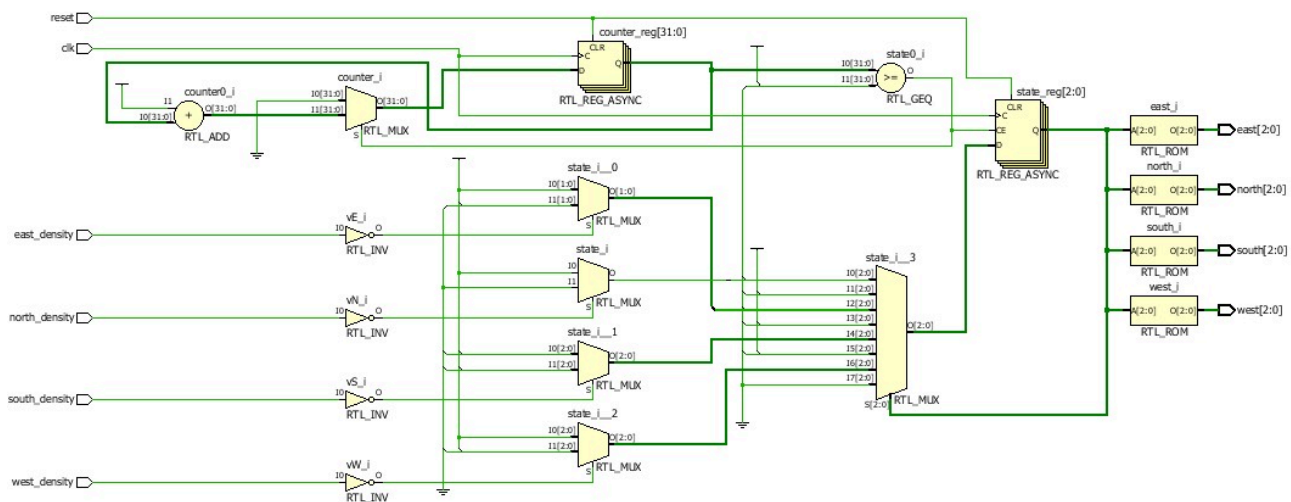


Fig. 3. RTL Schematic from Xilinx Vivado – showing counter, state register, comparator, MUX tree, and output ROM blocks

VIII. Simulation Results

A. Simulation Setup

The design was simulated in Xilinx Vivado using a TCL simulation script. The clock is driven at 100 MHz (10 ns period, 5 ns half-period). Two simulation runs were performed: Run 1 exercises normal operation for 10 μ s; Run 2 tests asynchronous reset behavior by asserting the reset signal.

B. Simulation Waveform

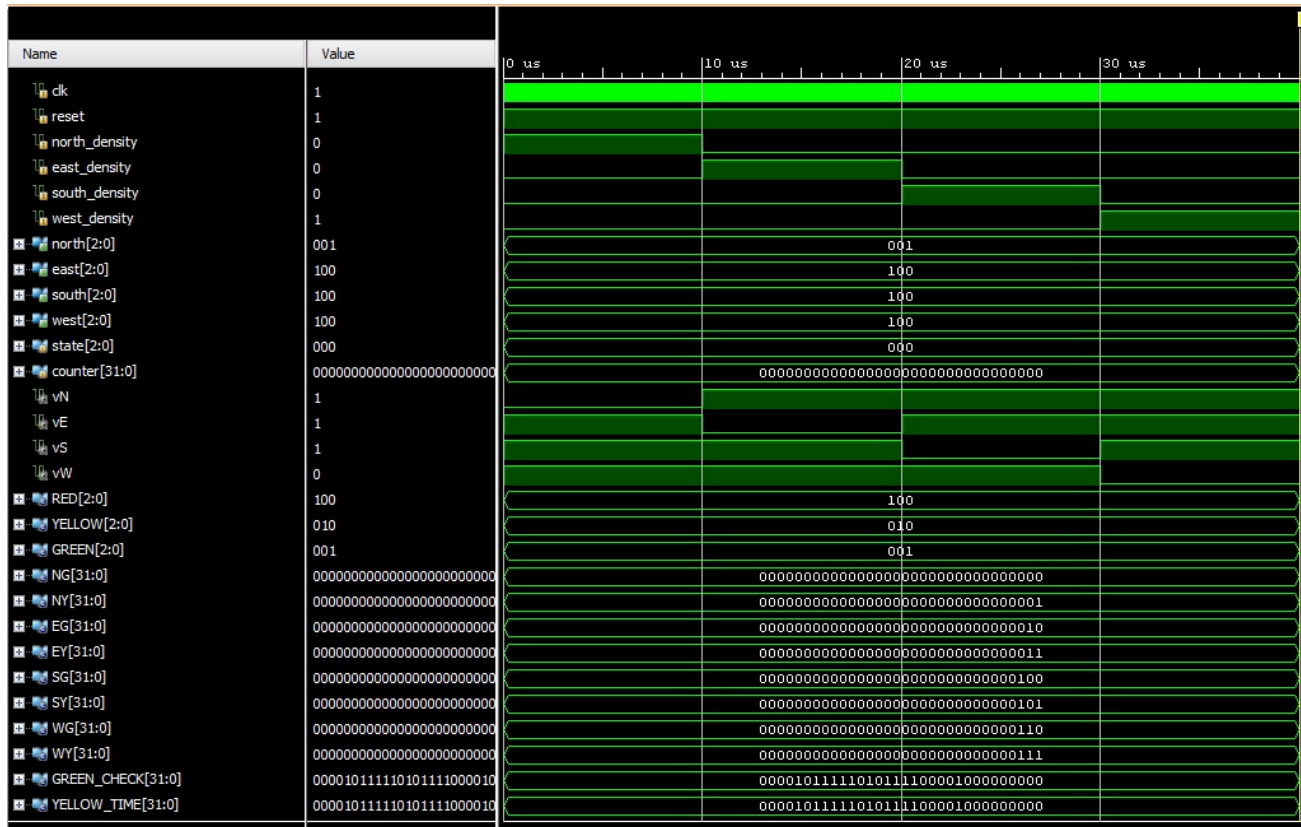


Fig. 4. Output Waveform from Xilinx Vivado – showing state transitions, counter values, and R/Y/G signal cycling

C. Verification Results

The simulation waveform confirms all key functional requirements:

- State Transitions: All 8 states (NG→NY→EG→EY→SG→SY→WG→WY→NG) transition correctly in sequence
- Phase Timing: Counter resets to zero at each state transition and counts up to GREEN_CHECK or YELLOW_TIME thresholds
- Zero Signal Conflicts: At all times, exactly one direction is non-RED; no two directions ever show Green simultaneously
- Output Encoding: north, east, south, west correctly output 001 (GREEN), 010 (YELLOW), or 100 (RED) per state
- Density Adaptation: With vW=0 (west sensor active), the WG state holds Green and resets the counter repeatedly
- Reset Behavior: Async reset immediately returns FSM to NG state and clears counter regardless of clock phase

D. TCL Simulation Script

The simulation is driven by a TCL script used in Vivado's Behavioral Simulation console. The script sets up a 100 MHz clock with add_force commands and runs two 10 μs simulation windows — one for normal

operation and one testing asynchronous reset. The structured TCL approach allows repeatable, automated verification.

TCL Simulation Script:-

restart

```
add_force {/Traffic_Light/clock} -radix bin {1 0ns} {0 5000ps} -repeat_every 10000ps
add_force {/Traffic_Light/reset} -radix bin {1 0ns}
add_force {/Traffic_Light/north_density} -radix bin {1 0ns}
add_force {/Traffic_Light/east_density} -radix bin {0 0ns}
add_force {/Traffic_Light/south_density} -radix bin {0 0ns}
add_force {/Traffic_Light/west_density} -radix bin {0 0ns}
run 10 us
```

```
add_force {/Traffic_Light/clock} -radix bin {1 0ns} {0 5000ps} -repeat_every 10000ps
add_force {/Traffic_Light/reset} -radix bin {1 0ns}
add_force {/Traffic_Light/north_density} -radix bin {0 0ns}
add_force {/Traffic_Light/east_density} -radix bin {1 0ns}
add_force {/Traffic_Light/south_density} -radix bin {0 0ns}
add_force {/Traffic_Light/west_density} -radix bin {0 0ns}
run 10 us
```

```
add_force {/Traffic_Light/clock} -radix bin {1 0ns} {0 5000ps} -repeat_every 10000ps
add_force {/Traffic_Light/reset} -radix bin {1 0ns}
add_force {/Traffic_Light/north_density} -radix bin {0 0ns}
add_force {/Traffic_Light/east_density} -radix bin {0 0ns}
add_force {/Traffic_Light/south_density} -radix bin {1 0ns}
add_force {/Traffic_Light/west_density} -radix bin {0 0ns}
run 10 us
```

```
add_force {/Traffic_Light/clock} -radix bin {1 0ns} {0 5000ps} -repeat_every 10000ps
add_force {/Traffic_Light/reset} -radix bin {1 0ns}
add_force {/Traffic_Light/north_density} -radix bin {0 0ns}
add_force {/Traffic_Light/east_density} -radix bin {0 0ns}
```

```
add_force {/Traffic_Light/south_density} -radix bin {0 0ns}
add_force {/Traffic_Light/west_density} -radix bin {1 0ns}
run 10 us
```

IX. Comparison with Existing Designs

Parameter	Manual Control	Timer-Based IC	This Design (FPGA)
Control Method	Human operator	555-timer ICs	FSM Verilog / FPGA
Flexibility	High (manual)	Fixed timing	Fully reprogrammable
Reliability	Low	Medium	High
Cost	High (labor)	Low	Medium
Scalability	Poor	Poor	Excellent
Emergency Override	Manual only	Not possible	Easily programmable
Hardware	Traffic booth	Discrete ICs	Single FPGA + PCB
Simulation	Not possible	Limited	Full waveform sim
Adaptive Timing	Manual	Not possible	Sensor-driven adaptive

Table III. Comparison of Traffic Light Control Approaches

The FPGA-based FSM design outperforms both manual and discrete IC approaches in reliability, scalability, and programmability. The ability to modify timing parameters, add emergency preemption logic, or extend to larger intersections purely through HDL changes without any hardware redesign is a critical advantage for real-world deployment.

X. Hardware Implementation

A. FPGA Deployment Steps

The complete hardware deployment workflow on the Basys 3 FPGA board follows six stages:

- Step 1 – Create Vivado Project: New RTL project targeting Artix-7 (XC7A35T) part on Basys 3
- Step 2 – Add Verilog Source: Add Traffic_Light.v and the XDC constraints file to the project
- Step 3 – Synthesis & Implementation: Run synthesis, place-and-route, and verify timing closure
- Step 4 – Generate Bitstream: Generate the .bit configuration file from the implemented design
- Step 5 – Program FPGA via USB: Load bitstream using Vivado Hardware Manager; SW0=reset, SW1=run
- Step 6 – Observe on PCB / LEDs: Verify that R/Y/G signals cycle correctly across all 4 junction directions

B. Hardware Prototype

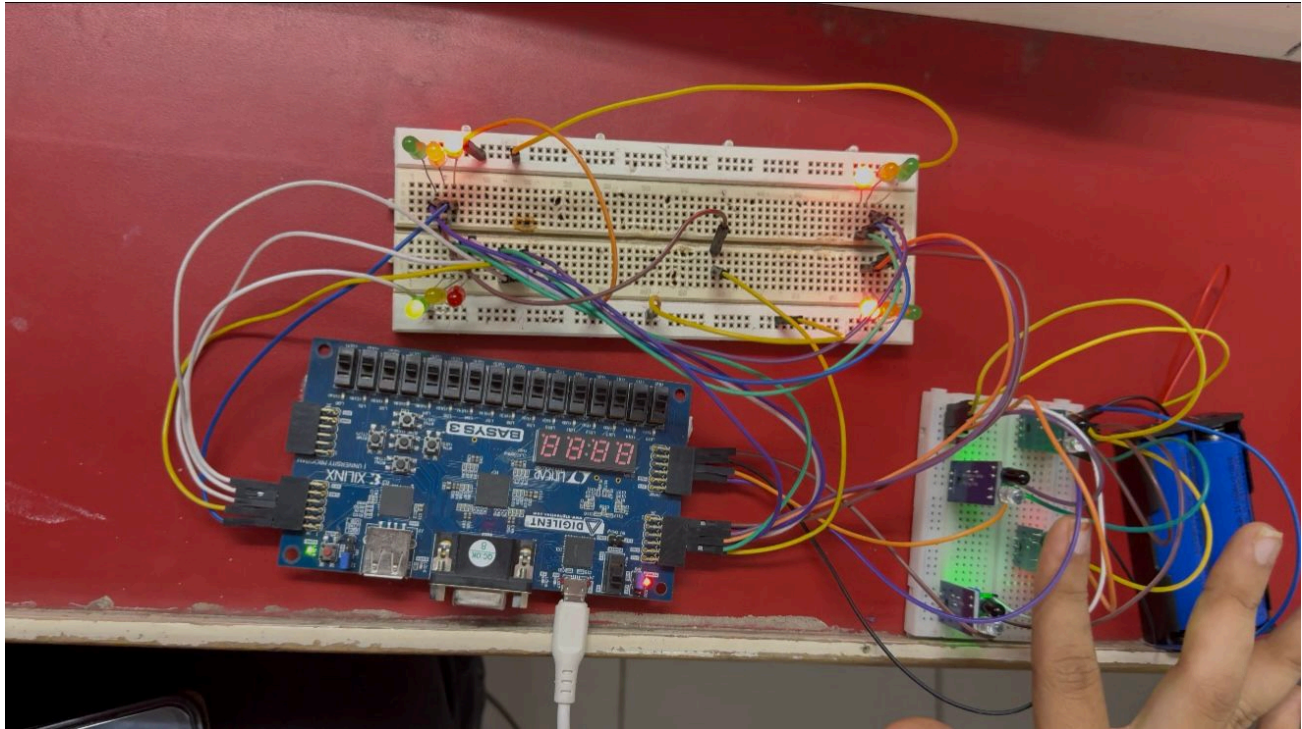


Fig. 5. Hardware Implementation – Basys 3 FPGA board connected to breadboard with LED traffic lights representing the 4-way junction

The hardware prototype uses the Basys 3 board's GPIO pins connected to a breadboard carrying 12 LEDs (3 per direction: Red, Yellow, Green). Each LED is protected by a current-limiting resistor. The custom PCB layout arranges the 4-direction traffic light clusters in a cross pattern physically representing the road junction. The design operates correctly in real-time, with each Green phase lasting 2 seconds and each Yellow phase also lasting 2 seconds at the 100 MHz clock rate.

XI. Conclusion

In this project, an Adaptive Traffic Light Controller for a 4-way road junction was successfully designed and implemented using a Moore FSM in Verilog HDL. The system correctly sequences traffic signals through 8 states — cycling Green and Yellow phases for each of the four cardinal directions — with adaptive timing driven by active-LOW traffic density sensors.

The design was verified through functional simulation in Xilinx Vivado, with waveforms confirming correct state transitions, proper phase timing, zero signal conflicts, and accurate adaptive hold behavior. Hardware deployment on the Basys 3 FPGA board (Artix-7 XC7A35T) demonstrated reliable real-time operation. A custom PCB physically modeling the 4-way junction completed the end-to-end demonstration.

The project demonstrates how FPGA-based FSM controllers surpass manual and timer-based approaches in reliability, scalability, and programmability, making them suitable for real-world smart-city traffic management applications.

Future enhancements may include: adaptive timing algorithms with variable Green/Yellow durations based on queue length measurements, emergency vehicle preemption logic using IR sensors, pedestrian crossing phase integration, multi-junction coordination over a network, and migration to larger FPGA platforms such as the Zynq UltraScale+ for embedded processing support.

Acknowledgment

The authors would like to thank the Department of Electronics and Communication Engineering, Sarvajani College of Engineering & Technology, Surat, for providing access to the Xilinx Vivado EDA tools and the Basys 3 FPGA development board. The authors also express sincere gratitude to Prof. (Dr.) Nehal Shah, Prof. Chintan Panchal, and Prof. (Dr.) Ketki Pathak for their guidance and support throughout this project.

References

- [1] S. Brown and Z. Vranesic, Fundamentals of Digital Logic with Verilog Design, 3rd ed. McGraw-Hill, 2014.
- [2] M. M. Mano and C. R. Kime, Logic and Computer Design Fundamentals, 4th ed. Pearson Education, 2008.
- [3] Xilinx Inc., Vivado Design Suite User Guide: Synthesis, UG901, 2023. [Online]. Available: <https://www.xilinx.com>
- [4] Digilent Inc., Basys 3 Reference Manual, 2023. [Online]. Available: <https://digilent.com>
- [5] V. A. Pedroni, Circuit Design and Simulation with VHDL, 2nd ed. MIT Press, 2010.
- [6] J. Bhasker, A Verilog HDL Primer, 3rd ed. Star Galaxy Publishing, 2005.